Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Artificial Intelligence Practical

Practical – VI

Roll No. : T045	Name: Omkar Thukarul
Class : TYIT	Date/Time of Submission : 1/08/2026

Aim: Machine Learning

1)Using any small dataset (e.g., Iris or a CSV dataset), write a Python program to demonstrate the basic machine learning workflow.
a. Load dataset using Pandas.
b. Perform basic preprocessing (handling missing values or scaling).
c. Split dataset into training and testing sets.
d. Train a simple classifier.
e. Display training and testing accuracy.

CODE:

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, MinMaxScaler, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load Dataset
df = pd.read_csv("iris - iris - iris - iris.csv")

# Display Dataset
print("First 5 Rows")
print(df.head())

# Check Dataset Information
print("\nDataset Information")
df.info()

# Display Dataset Shape
print("\nDataset Shape")
print(df.shape)

# Display Column Names
print("\nColumn Names")
print(df.columns)

# Display Summary Statistics
print("\nSummary Statistics")
print(df.describe())

# Check Data Types
print("\nData Types")
print(df.dtypes)
```

OUTPUT:

```
First 5 Rows
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa

Dataset Information
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal_length  150 non-null    float64
 1   sepal_width   150 non-null    float64
 2   petal_length  150 non-null    float64
 3   petal_width   150 non-null    float64
 4   species       150 non-null    str
dtypes: float64(4), str(1)
memory usage: 6.0 KB

Dataset Shape
(150, 5)

Column Names
Index(['sepal_length', 'sepal_width', 'petal_length', 'petal_width',
       'species'],
      dtype='str')
```

```
Summary Statistics
       sepal_length  sepal_width  petal_length  petal_width
count    150.000000   150.000000    150.000000   150.000000
mean       5.843333     3.054000      3.758667     1.198667
std        0.828066     0.433594      1.764420     0.763161
min        4.300000     2.000000      1.000000     0.100000
25%        5.100000     2.800000      1.600000     0.300000
50%        5.800000     3.000000      4.350000     1.300000
75%        6.400000     3.300000      5.100000     1.800000
max        7.900000     4.400000      6.900000     2.500000

Data Types
sepal_length     float64
sepal_width      float64
petal_length     float64
petal_width      float64
species              str
dtype: object
```

```
# Check Missing Values
print("\nMissing Values")
print(df.isnull().sum())

# Handle Missing Values
df.fillna(df.mean(numeric_only=True), inplace=True)
print("\nMissing Values Handled")

# Remove Duplicate Records
df.drop_duplicates(inplace=True)
print("Duplicates Removed")

# Encode Categorical Data
label = LabelEncoder()
df["species"] = label.fit_transform(df["species"])
print("Categorical Data Encoded")

# Rename Columns
df.rename(columns={
    "sepal_length": "Sepal_Length",
    "sepal_width": "Sepal_Width",
    "petal_length": "Petal_Length",
    "petal_width": "Petal_Width"
}, inplace=True)

print("Columns Renamed")

# Drop Unnecessary Columns
print("No Unnecessary Columns to Drop")

# Select Features
X = df.drop("species", axis=1)
y = df["species"]

print("Features Selected")

# Feature Engineering
X["Flower_Size"] = X["Sepal_Length"] + X["Petal_Length"]

print("New Feature Created")

# Convert Data Types
X = X.astype(float)

print("Data Types Converted")

# Normalize Data
minmax = MinMaxScaler()
X_normalized = minmax.fit_transform(X)

print("Min-Max Scaling Completed")

# Standardize Data
standard = StandardScaler()
X_standardized = standard.fit_transform(X)
```

```
Missing Values
sepal_length    0
sepal_width     0
petal_length    0
petal_width     0
species         0
dtype: int64

Missing Values Handled
Duplicates Removed
Categorical Data Encoded
Columns Renamed
No Unnecessary Columns to Drop
Features Selected
New Feature Created
Data Types Converted
Min-Max Scaling Completed
Standard Scaling Completed
```

```
print("Standard Scaling Completed")

# Split Features and Target Variable
X_train, X_test, y_train, y_test = train_test_split(
    X_standardized,
    y,
    test_size=0.2,
    random_state=42
)

print("\nTraining Shape:", X_train.shape)
print("Testing Shape:", X_test.shape)

# Train Classifier
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

print("\nModel Trained Successfully")

# Display Training Accuracy
train_pred = model.predict(X_train)
train_accuracy = accuracy_score(y_train, train_pred)

print("Training Accuracy:", train_accuracy)

# Display Testing Accuracy
test_pred = model.predict(X_test)
test_accuracy = accuracy_score(y_test, test_pred)

print("Testing Accuracy:", test_accuracy)
```

```
Training Shape: (117, 5)
Testing Shape: (30, 5)

Model Trained Successfully
Training Accuracy: 1.0
Testing Accuracy: 0.9333333333333333
```

Example Data

CODE:

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, MinMaxScaler, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load Dataset
df = pd.read_csv("Titanic-Dataset.csv")

# Display Dataset
print("First 5 Rows")
print(df.head())

# Check Dataset Information
print("\nDataset Information")
df.info()

# Display Dataset Shape
print("\nDataset Shape")
print(df.shape)
```

OUTPUT:

```
First 5 Rows
   PassengerId  Survived  Pclass                                               Name     Sex   Age  SibSp  Parch            Ticket     Fare Cabin Embarked
0            1         0       3                            Braund, Mr. Owen Harris    male  22.0      1      0         A/5 21171   7.2500   NaN        S
1            2         1       1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1      0          PC 17599  71.2833   C85        C
2            3         1       3                             Heikkinen, Miss. Laina  female  26.0      0      0  STON/O2. 3101282   7.9250   NaN        S
3            4         1       1       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1      0            113803  53.1000  C123        S
4            5         0       3                           Allen, Mr. William Henry    male  35.0      0      0            373450   8.0500   NaN        S
```

```
# Display Column Names
print("\nColumn Names")
print(df.columns)

# Display Summary Statistics
print("\nSummary Statistics")
print(df.describe())

# Check Data Types
print("\nData Types")
print(df.dtypes)

# Check Missing Values
print("\nMissing Values")
print(df.isnull().sum())

# Handle Missing Values
df["Age"].fillna(df["Age"].mean(), inplace=True)
df["Embarked"].fillna(df["Embarked"].mode()[0],
 inplace=True)

if "Cabin" in df.columns:
    df.drop("Cabin", axis=1, inplace=True)

print("\nMissing Values Handled")

# Remove Duplicate Records
df.drop_duplicates(inplace=True)
print("Duplicates Removed")

# Encode Categorical Data
label = LabelEncoder()

df["Sex"] = label.fit_transform(df["Sex"])
df["Embarked"] = label.fit_transform(df["Embarked"])

print("Categorical Data Encoded")

# Rename Columns
df.rename(columns={
    "Pclass": "Passenger_Class",
    "SibSp": "Siblings_Spouses",
    "Parch": "Parents_Children"
}, inplace=True)

print("Columns Renamed")

# Drop Unnecessary Columns
drop_columns = ["PassengerId", "Name", "Ticket"]

for col in drop_columns:
    if col in df.columns:
        df.drop(col, axis=1, inplace=True)

print("Unnecessary Columns Dropped")
```

```
Dataset Information
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    str
 4   Sex          891 non-null    str
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    str
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    str
 11  Embarked     889 non-null    str
dtypes: float64(2), int64(5), str(5)
memory usage: 83.7 KB

Dataset Shape
(891, 12)

Column Names
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
       'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],
      dtype='str')
```

```
Summary Statistics
       PassengerId    Survived      Pclass         Age       SibSp       Parch        Fare
count   891.000000  891.000000  891.000000  714.000000  891.000000  891.000000  891.000000
mean    446.000000    0.383838    2.308642   29.699118    0.523008    0.381594   32.204208
std     257.353842    0.486592    0.836071   14.526497    1.102743    0.806057   49.693429
min       1.000000    0.000000    1.000000    0.420000    0.000000    0.000000    0.000000
25%     223.500000    0.000000    2.000000   20.125000    0.000000    0.000000    7.910400
50%     446.000000    0.000000    3.000000   28.000000    0.000000    0.000000   14.454200
75%     668.500000    1.000000    3.000000   38.000000    1.000000    0.000000   31.000000
max     891.000000    1.000000    3.000000   80.000000    8.000000    6.000000  512.329200
```

```
Data Types
PassengerId      int64
Survived         int64
Pclass           int64
Name               str
Sex                str
Age              float64
SibSp            int64
Parch            int64
Ticket             str
Fare             float64
Cabin              str
Embarked           str
dtype: object
```

```
# Select Features
X = df.drop("Survived", axis=1)
y = df["Survived"]

print("Features Selected")

# Feature Engineering
if "Siblings_Spouses" in X.columns and "Parents_Children" in X.columns:
    X["Family_Size"] = (
        X["Siblings_Spouses"] +
        X["Parents_Children"] + 1
    )

print("New Feature Created")

# Convert Data Types
X["Age"] = X["Age"].astype(float)

print("Data Types Converted")

# Normalize Data
minmax = MinMaxScaler()
X_normalized = minmax.fit_transform(X)

print("Min-Max Scaling Completed")

# Standardize Data
standard = StandardScaler()
X_standardized = standard.fit_transform(X)

print("Standard Scaling Completed")

# Split Features and Target Variable
X_train, X_test, y_train, y_test = train_test_split(
    X_standardized,
    y,
    test_size=0.2,
    random_state=42
)

print("\nTraining Shape:", X_train.shape)
print("Testing Shape:", X_test.shape)

# Train Classifier
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

print("\nModel Trained Successfully")

# Display Training Accuracy
train_pred = model.predict(X_train)
train_accuracy = accuracy_score(y_train, train_pred)

print("Training Accuracy:", train_accuracy)

# Display Testing Accuracy
```

```
Missing Values
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

```
Missing Values Handled
Duplicates Removed
Categorical Data Encoded
Columns Renamed
Unnecessary Columns Dropped
Features Selected
New Feature Created
Data Types Converted
Min-Max Scaling Completed
Standard Scaling Completed

Training Shape: (712, 8)
Testing Shape: (179, 8)

Model Trained Successfully
Training Accuracy: 0.9803370786516854
Testing Accuracy: 0.8044692737430168
```

```
test_pred = model.predict(X_test)
test_accuracy = accuracy_score(y_test, test_pred)

print("Testing Accuracy:", test_accuracy)
```